

STRATÉGIAI TERMÉK ÉS PIACI INTELLIGENCIA

TOP 10 ÉRTÉKNÖVELŐ NÖVEKEDÉSI STRATÉGIA

Hogyan válik a Panellakó a CEE társasházi piac #1 PropTech platformjává

Társasházi digitális működési központ — PropTech SaaS magyarországi és CEE panel-épületekhez

€2.66M-€7.09M

Célértékelés

3-5x

Értékszorzó

10

Növekedési Kezdeményezés

Panellakó — Társasházi digitális működési központ — PropTech SaaS magyarországi és CEE panel-épületekhez

Készítve: 2026-05-23 | Bizalmas

Kiindulási értékelés: €400k-€2.2M → Célértékelés: €2.66M-€7.09M

VEZETŐI ÖSSZEFOGLALÓ

Mit tartalmaz a jelentés

Egy rangsorolt készlet a(z) Panellakó számára magas hatékonyságú növekedési kezdeményezésekből, mindegyik várható értéknövekedési hatással értékelve, kész-beilleszthető megvalósítási prompttal és iteratív mélyítésre szolgáló újragenerálási prompttal.

Hogyan használjuk

Az utolsó oldalon található értékmátrix nyújt áttekintést. Minden rangsorolt elem önállóan értelmezhető — válasszon bármelyiket részletes elemzéshez. A Megvalósítási Prompt szekciók közvetlen beillesztésre vannak megtervezve egy AI kódoló asszisztensbe a kódbázisban.

Módszertan: piaci szorzók nyilvános értékelési adatbázisokból (PitchBook, Crunchbase, SaaS Capital Index), versenytárs árazási oldalakról és elemzői jelentésekből (Gartner, IDC, Forrester). Az értéknövekedési hatás tartomány-alapú, a helyettesítési költség, az összehasonlítható ARR-szorzó és a stratégiai prémium szempontjából háromszögelt.

TARTALOMJEGYZÉK

#	Kezdeményezés	Becsült Hatás
1	Multi-épület portfólió dashboard — Közös képviselői skálázási architektúra	+€450k-€900k
2	Teljes Stripe előfizetési életciklus — Próba → Fizetős → Lejárt → Lemondás	+€380k-€800k
3	AI hibabejelentés triage + kivitelező irányítás — Versenyelőny Claude API-val	+€320k-€680k
4	Automatizált közgyűlési jegyzőkönyv generátor — Közgyűlési Jegyzőkönyv PDF	+€250k-€550k
5	Teljes pénzügyi főkönyv — Kettős könyvviteli közös költség könyvelés	+€220k-€480k
6	Tranzakciós e-mail csomag Resend-del — Teljes kommunikációs életciklus	+€180k-€400k
7	Környezeti intelligencia dashboard — SEO-ból termékkonverzió motor	+€150k-€340k
8	SSR auth keményítés + Middleware route védelem	+€130k-€300k
9	Lakói önkiszolgáló portál — Mobil PWA + Push értesítés mélyítés	+€100k-€240k
10	PostHog termék-analitika — Konverziós tölcsér + funkció-használat instrumentálás	+€80k-€200k

TELJES KOMBINÁLT POTENCIÁL

+€2.26M-€4.89M

Multi-épület portfólió dashboard — Közös képviselői skálázási architektúra

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A Panellakó legtöbb bevételt generáló növekedési lépése a professzionális közös képviselői és ügynökségi szegmens teljes feloldása. A munkaterület-routing már `/w/[buildingId]` alakot követ (a v3.16.0 governance óta kötelező), és a `components/workspace-shell.tsx` létezik, de a portfólió-szintű intelligencia — összesített hibabejelentési sor, több épületre kiterjedő pénzügyi áttekintés, épületek közötti összehasonlítás — még nem jelenik meg. A professzionális kezelők cégenkénti 8-25 épületet kezelnek; a portfólió dashboard ugyanolyan szorzóval növeli az ügyfélenkénti bevételt.

Magyarországon körülbelül 2 400 engedéllyel rendelkező ingatlankezelő cég működik, amelyek átlagosan 8-25 épületet kezelnek. Egyetlen ügynökség belépése Enterprise szinten 5 760-18 000 € éves ARR-t jelent a jelenlegi Pro árazással (3 €/albetét/hó × 40 átlagos albetét × 12 hónap × 8-25 épület). Az OnlineHáz épületenként szolgál ki, portfólió nézetük nincs. A Domus24 csak alap épületlistát nyújt, több épületes elemzést nem. A Panellakó mindkettőt megelőzheti egy valódi portfólió-intelligencia réteggel.

Technikai megközelítés: `/app` épületválasztó hozzáadása (az `app/app/page.tsx` épületlistát jelenít meg a `memberships` RLS-szűrt lekérdezéséből) és portfólió összefoglaló oldal a választó szintjén. Aggregálás meglévő Supabase táblákból: nyitott hibabejelentések épületenként, összes hátralék, közelgő közgyűlési dátumok, környezeti pontszám. Recharts kereszt-épület összehasonlító oszlopdiagramokkal.

3. PONT — MEGVALÓSÍTÁSI PROMPT

- Hozd létre: `app/app/page.tsx` — lekérdezi a `public.get\_my\_buildings()` RPC-t (migráció: `20260516\_get\_my\_buildings\_rpc.sql`), BuildingCard rácsot jelenít meg albetét számmal, nyitott hibabejelentés jelvénnel, hátralék indikátorral.
- Hozd létre: `app/app/portfolio/page.tsx` — összesített KPI-ok: összes nyitott hibabejelentés, összes kintlévőség, lejárt közös kötelezettségű épületek, közelgő közgyűlések 30 napon belül.
- Adj hozzá `components/portfolio-stats-bar.tsx` komponens: Recharts BarChart, épületek összehasonlítás a megoldatlan hibabejelentések, hátralék és env pontszám alapján.
- Frissítsd a `components/workspace-sidebar.tsx`-t: 'Portfólió áttekintése' link a nav tetején, 'Vissza az összes épülethez' morzsa-navigáció.
- Védd az `app/app/\*\*` route-okat a `middleware.ts`-ben.
- Adj hozzá `portfolio\_role` oszlopot a `building\_memberships` táblához (ügynökség vs. egyéni képviselő) az upsell üzenetek vezérléséhez.
- Hozd létre: `app/actions/portfolio.ts` — `getPortfolioSummary(userId)` Server Action épület-aggregátum okkal.
- Kösd be a Stripe számlázási oldalt (`app/billing/billing-client.tsx`) 'Ügynökségi' multi-épületes szint ajánlással.

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Feloldott ügyfélszegmens	Egyépületes kezelők → Portfóliókezelők (8-25 épület)
ARR szorzó ügyfélenként	720-1 440 €/év → 5 760-36 000 €/év
Magyarországi piac	~2 400 ingatlankezelő cég
Értékelési hatás	+450 000-900 000 € (15-25× ARR szorzón)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€450k-€900k**5. PONT — ÚJRAGENERÁLÁSI PROMPT**

Senior Next.js 14 + Supabase fejlesztőként tervezd meg és valósítsd meg a Panellakó portfólió dashboardját. A termék v0.9.23-on van, a repo: /home/user/panellako. Meglévő: `app/w/[buildingId]/(subpages)/` route tree, `public.get\_my\_buildings()` RPC (`20260516\_get\_my\_buildings\_rpc.sql`), `components/workspace-sidebar.tsx`. Tervezd meg: (1) `app/app/page.tsx` épületválasztó KPI jelvényekkel, (2) `app/app/portfolio/page.tsx` összesített dashboard, (3) Recharts kereszt-épület összehasonlítás, (4) sidebar morzsa-navigáció, (5) Stripe multi-épületes szint. Teljes TypeScript kód, Supabase lekérdezési minták, RLS megfontolások.

PANG 2
Teljes Stripe előfizetési életciklus — Próba → Fizetős → Lejárt → Lemondás

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A PanelLakóba integrált a Stripe Checkout (``app/api/stripe/checkout/route.ts``, ``app/api/stripe/webhook/route.ts``, ``app/api/stripe/portal/route.ts``) és van számlázási UI (``app/billing/billing-client.tsx``), de az előfizetési életciklus nem teljes: nincs automatikus trial lejárat kényszerítés, nincs lejárt fizetési dunning folyamat, nincs szint-váltási út, és nincs lemondáshoz kötött visszanyerési sorozat. Egy lyukas számlázási tölcser azt jelenti, hogy minden lemorzsolódott épület végleges bevételkiesés.

SaaS számlázási benchmarkok (Stripe 2025 State of Subscriptions): automatizált dunninggal rendelkező termékek az akaratlan lemorzsolódók 15-25%-át visszanyerik. Automatizált trial-to-paid konverzió ösztönzők (7. nap, 14 napos trial 12. napja) 18-30%-kal növelik a konverziót. A magyarországi piac sajátossága: a közös képviselők sokszor köztisztviselők vagy nyugdíjasok alacsony technikai jártassággal — a súrlódásmentes Stripe Ügyfélportál magyarul kulcsfontosságú megtartási eszköz.

Technikai megközelítés: a meglévő ``20260516_billing.sql`` migrációt bővíteni kell egy ``tenant_subscriptions`` tábla triggerrel, amely Resend e-maileket küld a próba 7. napján, 12. napján (konverzió ösztönző), az 1. lejárt napon (dunning) és a 15. napon (végső értesítés felfüggesztés előtt). Stripe webhooks: ``customer.subscription.trial_will_end``, ``invoice.payment_failed``, ``customer.subscription.deleted``.

3. PONT — MEGVALÓSÍTÁSI PROMPT

```
1. Bővítsd a `supabase/migrations/20260516_billing.sql` migrációt: adj hozzá `trial_ends_at`, `overdue_since`, `cancellation_requested_at` oszlopokat a `tenant_subscriptions`-hoz.
2. `app/api/stripe/webhook/route.ts`-ben: kezelje a `customer.subscription.trial_will_end`-et → Resend trial ösztönző e-mail; `invoice.payment_failed` → `overdue_since` beállítás, dunning e-mail.
3. Hozd létre: `app/actions/billing.ts` – `enforceTrialGate(buildingId)` ellenőrzi a `tenant_subscriptions`-t, visszaadja `{ allowed: boolean, daysLeft: number }`.
4. `middleware.ts`-ben: `app/w/[buildingId]/**` route-oknál hívd az `enforceTrialGate`-t; átirányítás `app/billing/page.tsx`-re ha lejárt.
5. Frissítsd az `app/billing/billing-client.tsx`-t: trial visszaszámláló sáv, lejárt fizetési figyelmeztető sáv, szint-váltó kártyák.
6. Hozd létre e-mail sablonokat: `lib/email-templates/billing/trial-nudge.tsx`, `overdue-notice.tsx`, `cancellation-confirmation.tsx`.
7. Adj hozzá PostHog eseményeket: `trial_started`, `trial Converted`, `payment_failed`, `subscription_cancelled`.
```

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Trial-to-paid konverzió növekedés	+18-30% automatizált ösztönzőkkel
Akaratlan lemorzsolódás visszanyerése	A sikertelen fizetések 15-25%-a visszanyerhető
Bevételi láthatóság	Teljes ARR/MRR/churn dashboard Stripe-ban
Értékelési hatás	+380 000-800 000 € (számlázási infrastruktúra = 2-3× ARR szorzó prémium)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€380k-€800k**5. PONT — ÚJRAGENERÁLÁSI PROMPT**

Senior full-stack fejlesztőként tervezd meg a Panellakó (v0.9.23, /home/user/panellako) teljes Stripe előfizetési életciklusát. Meglévő: `app/api/stripe/checkout/route.ts`, `app/api/stripe/webhook/route.ts`, `app/api/stripe/portal/route.ts`, `app/billing/billing-client.tsx`, `supabase/migrations/20260516\_billing.sql`, `lib/email.ts` (Resend). Tervezd meg: (1) trial visszaszámláló kényszerítés middleware.ts-ben, (2) webhook kezelők, (3) Resend dunning e-mail sablonok, (4) billing-client.tsx overdue/upgrade UI, (5) PostgreSQL tölcser események.

AI hibabejelentés triage + kivitelező irányítás — Versenyelőny Claude API-val

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A Panellakóban már létezik a ``supabase/functions/triage-ticket`` Edge Function könyvtár és az ``app/actions/tickets.ts`` Server Action réteg. A jelenlegi állapot egy proof-of-concept: az AI triage fut, de az eredmények nem kapcsolódnak automatizált kivitelező irányításhoz, prioritás eszkalációs folyamatokhoz vagy a lakói kommunikációs körhöz. Ennek a folyamatnak a befejezése — a triage kimenettől a kivitelező küldési javaslaton át az automatizált lakói állapotfrissítésig — valódi workflow-automatizálási mozatot hoz létre.

A globális PropTech AI piac 2030-ra várhatóan eléri a 41,5 milliárd dollárt (CBRE Tech Report 2025). A lakóingatlan-kezelési szegmensben a legfontosabb megtérülési mutatószám a 'ticket-to-resolution time' csökkentése. Egy 15 épületet kezelő, 50+ albetétes képviselő heti 10-20 hibabejelentést dolgoz fel; az AI triage + kivitelező irányítás heti 3-5 órát takarít meg — ez évi 3 750-10 400 € időmegtakarítást jelent ügyfelenként.

Technikai megközelítés: a ``supabase/functions/triage-ticket/index.ts`` kibővítése ``claude-haiku-4-5`` modellel strukturált `tool_use` hívással, amely visszaadja a ``{category, urgency_1_to_10, vendor_type, estimated_cost_range_huf, summary_hu, resident_update_hu}`` kimenetet. A kimenet bekötése: (1) ``tickets.ai_category`` + ``tickets.ai_urgency`` automatikus frissítése, (2) ``work_order`` sor létrehozása, (3) lakói push értesítés küldése a meglévő ``supabase/functions/send-push`` funkcióval.

3. PONT — MEGVALÓSÍTÁSI PROMPT

```
1. Migrációs szkript: `ALTER TABLE tickets ADD COLUMN ai_category TEXT, ADD COLUMN ai_urgency INT, ADD COLUMN ai_vendor_suggestion TEXT, ADD COLUMN ai_summary TEXT, ADD COLUMN ai_resident_update TEXT;`
2. Bővítsd a `supabase/functions/triage-ticket/index.ts`-t: Anthropic `claude-haiku-4-5` tool_use-szal; kimenet: `{category, urgency, vendor_type, estimated_cost_range_huf, summary_hu, resident_update_hu}`.
3. `app/actions/tickets.ts` `createTicket()`-ben: az insert után fire-and-forget hívás: `supabase.functions.invoke('triage-ticket', ...)` (nincs await).
4. Hozd létre: `app/actions/work-orders.ts` `createWorkOrderFromTriage(ticketId)` : `ai_vendor_suggestion` alapján `work_orders` sort hoz létre.
5. Kösd be a `send-push`-t: `ai_resident_update` szöveg küldése a bejelentő push előfizetésébe.
6. Ticket lista UI-ban: urgency szín jelvény (zöld/sárga/piros), vendor-type chip.
7. Adj hozzá `ANTHROPIC_API_KEY`-t az Edge Function secrets-be.
8. PostHog esemény: `ticket_ai_triage_completed` analyticshez.
```

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Képviselői időmegtakarítás	3-5 óra/hét portfólión (10+ épület)
Ticket-to-resolution idő	-30-40% gyorsabb kivitelező irányítás
AI termék prémium	Első AI-natív PropTech Magyarországon — 1,5-2× ARR szorzó növekedés
Kiegészítő bevételi potenciál	15-30 €/hó/épület AI-szint frissítés

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€320k-€680k

5. PONT — ÚJRAGENERÁLÁSI PROMPT

Senior Supabase Edge Functions + Anthropic API fejlesztőként valósítsd meg a Panellakó AI triage pipeline-ját. Meglévő: `supabase/functions/triage-ticket/` könyvtár, `supabase/functions/send-push/` függvény, `app/actions/tickets.ts`. Valósítsd meg: (1) triage-ticket bővítése claude-haiku-4-5 tool_use-szal, (2) migráció az ai_* oszlopok hozzáadásához, (3) fire-and-forget hívás a createTicket Server Actionban, (4) work-order automatikus létrehozás, (5) push értesítés lakóknak, (6) urgency badge UI.

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A PanelLakóban már létezik a `supabase/functions/generate-assembly-protocol` Edge Function könyvtár, a szavazási modul és az `app/actions/meetings.ts` Server Action. A `@react-pdf/renderer` csomag hivatkozott a kódbázisban. Ami hiányzik: a közgyűlés lezárása → protokoll generálás → dokumentumtárolás → e-mail kézbesítés pipeline. Ez egy compliance-vezérelt, magas észlelt értékű funkció: minden magyarországi társasház Ptk. 5:85-5:88 alapján kötelezett aláírt közgyűlési jegyzőkönyvet készíteni 15 napon belül.

A közös képviselők Magyarországon közgyűlésenként 2-4 órát töltenek a Közgyűlési Jegyzőkönyv Word-ben való elkészítésével, manuálisan beírva a jelenléti listát, a határozatok szövegét és a szavazati számokat. A PanelLakó ezt 1 kattintásos, azonnal jogilag megfelelő PDF generálásra csökkentheti. Hasonló automatizálás létezik a vállalati irányítási SaaS szektorban (Board Intelligence, Diligent Boards 5-15 €/ülés/hó áron). Egyetlen magyarországi PropTech versenytárs sem kínál ilyet.

Technikai megközelítés: a `generate-assembly-protocol` Edge Function kiolvassa a meeting + agenda_items + resolutions + votes + building_members adatokat Supabase-ből, `@react-pdf/renderer`-rel Ptk-kompatibilis PDF sablont renderel, feltölti Supabase Storage-ba, és Resend e-mailt küld a közös képviselőnek az aláírt letöltési URL-lel.

3. PONT — MEGVALÓSÍTÁSI PROMPT

1. Migráció: `ALTER TABLE meetings ADD COLUMN status TEXT DEFAULT 'tervezett' CHECK (status IN ('tervezett', 'aktív', 'lezarva'))`;
2. `app/actions/meetings.ts` `closeAssembly(meetingId, buildingId)` beállítja `status = 'lezarva'`, meghívja a `generate-assembly-protocol` Edge Functiont.
3. `supabase/functions/generate-assembly-protocol/index.ts` kibővítése: meeting + `agenda\_items` + `resolutions` + `votes` + `building\_members` lekérése; `@react-pdf/renderer`-rel PDF renderelés.
4. PDF szekciók: Épület adatai, Időpont/helyszín/összehívó; Jelenléti ív; Határozatképesség; Napirendi pontok szavazási eredményekkel; Határozatok szövege sorszámmal; Aláírás blokk.
5. Feltöltés Supabase Storage-ba: `documents/assembly-protocols/{buildingId}/{meetingId}.pdf`.
6. `documents` tábla sor insert: kategória: `kozgyulesi\_jkv`.
7. Resend e-mail a közös képviselőnek aláírt letöltési URL-lel.
8. 'Közgyűlés lezárása és Jegyzőkönyv generálása' gomb hozzáadása a közgyűlés részletes nézetéhez határozatképességi megerősítő párbeszéddel.

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Képviselői időmegtakarítás	2-4 óra/közgyűlés → <5 perc
Jogi megfelelés automatizálás	Ptk. 5:85-5:88 kompatibilis 1 kattintással
Funkció észlelt értéke	Top-3 legtöbbször kért funkció a magyar épületkezelésben
Pro szint igazolása	Egyértelműen differenciálja a Pro szintet az Alaphoz képest

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€250k-€550k

5. PONT — ÚJRAGENERÁLÁSI PROMPT

Senior Supabase + React PDF fejlesztőként valósítsd meg a teljes közgyűlés-lezárás → protokoll-generálás pipeline-t a Panellakóban (v0.9.23, /home/user/panellako). Meglévő: `supabase/functions/generate-assembly-protocol/` könyvtár, `app/actions/meetings.ts`, `@react-pdf/renderer`, Supabase Storage (`documents` bucket, `20260516\_create\_documents\_bucket.sql`), `lib/email.ts`. Valósítsd meg: (1) `meetings.status` migráció, (2) `closeAssembly` Server Action, (3) teljes Edge Function AssemblyProtocolTemplate komponenssel, (4) Storage feltöltés, (5) documents tábla insert, (6) Resend e-mail.

PANG 5 Teljes pénzügyi főkönyv — Kettős könyvviteli közös költség könyvelés

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A Panellakóban van `app/actions/finance.ts` Server Action és pénzügyi modul UI, de a jelenlegi implementáció hiányzik: kettős könyvviteli alapelvek, teljes közös költség generálási workflow (havi tömeges befizetési felszólítás az összes albetétnek), automatizált hátralékszámítás konfigurálható türelmi időkkal, és megfelelő éves pénzügyi kimutatás export. A magyarországi társasházi könyvelést a Lakástörvény (2003. évi CXXXIII.) §24 szabályozza, amely minden épületet kötelez a megfelelő pénzügyi nyilvántartás fenntartására és éves kimutatás nyújtására a tulajdonosoknak.

A magyarországi közös költség könyvelési piacot jelenleg Excel-táblázatok és örökség szoftverek uralják (pl. Társasházkezelő 2000 — egy Windows 95-os kori alkalmazás, amelyet még mindig széles körben használnak). Egy modern, jogilag megfelelő főkönyv a Panellakóban — amely az official 'Közös Költség Kimutatás' nyomtatvány formátumát generálja — pótolhatatlan workflow-függőséget hoz létre. Az épületeket kezelő könyvelők különálló persona (lásd `app/funkciok/konyveloknak/page.tsx`).

Technikai megközelítés: a `finance.ts` Server Actions kibővítése tömeges `generateMonthlyCharges()` akcióval, `unit\_ledger\_view` materializált nézet hozzáadása Supabase-ben, és PDF export `@react-pdf/renderer`-rel a Lakástörvény-kompatibilis éves 'Közös Költség Kimutatáshoz'.

3. PONT — MEGVALÓSÍTÁSI PROMPT

1. Migráció: `financial\_transactions` tábla létrehozása `(id, building\_id, unit\_id, type: 'charge'|'payment'|'adjustment', amount\_huf, description, period\_month, created\_by, created\_at)` oszlopokkal.
2. `unit\_ledger\_view` létrehozása Supabase-ben: albetétenként futó egyenleg `SUM(charges) - SUM(payments)` alapján.
3. `app/actions/finance.ts` bővítése: `generateMonthlyCharges(buildingId, month, chargePerUnit)` – tömeges common cost sorok az összes albetéthez egy tranzakcióban.
4. `recordPayment(unitId, amountHuf, paymentDate, payerName)` Server Action hozzáadása.
5. `getArrearsReport(buildingId)` Server Action: `balance < 0` albetétek `days\_overdue` kiszámítással.
6. PDF export: `generateKozosKoltsegKimutasas(buildingId, year)` – Lakástörvény-kompatibilis éves albetétenküli kimutatás `@react-pdf/renderer`-rel.
7. Pénzügyi dashboard: `unit\_ledger\_view` adatok rendezhető táblázatban piros/zöld egyenleg indikátorokkal; 'Havi közös költség generálás' varázsló.
8. Hátralék eszkaláció: >3 albetét >60 napos hátralék esetén manager push értesítés.

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Excel-helyettesítési ragadóság	Workflow lock-in: az épületek véglegesen elhagyják az Excelt
Könyvelői persona megtartás	3-5 óra/épület/hó megtakarítás éves egyeztetéskor
Megfelelőségi érték	Lakástörvény §24 kompatibilis éves kimutatás 1 kattintással
Értékelési hatás	+220 000-480 000 € (missziókritikus workflow = alacsony lemorzsolódás)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€220k-€480k

5. PONT — ÚJRAGENERÁLÁSI PROMPT

Senior Next.js 14 + Supabase pénzügyi rendszerek fejlesztőként valósítsd meg a teljes pénzügyi főkönyvet a Panellakóban (v0.9.23, /home/user/panellako). Meglévő: `app/actions/finance.ts`, `app/w/[buildingId]/(subpages)/` route tree, `@react-pdf/renderer`, Supabase migrációk. Valósítsd meg: (1) `financial\_transactions` tábla migráció, (2) `unit\_ledger\_view` materializált nézet, (3) `generateMonthlyCharges` tömeges Server Action, (4) `recordPayment`, (5) `getArrearsReport` days_overdue-val, (6) `generateKozosKoltsegKimutatas` PDF export, (7) pénzügyi dashboard UI rendezhető főkönyv táblázattal.

PANG 6

Tranzakciós e-mail csomag Resend-del — Teljes kommunikációs életciklus

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A PanelLakóban van ``lib/email.ts`` Resend-alapú ``sendEmail`` funkcióval és e-mail sablonok könyvtárral. A Supabase ``notifications`` táblájában van ``channel`` mező, amely 'email' értéket is támogat, de a legtöbb értesítési út csak alkalmazáson belüli vagy push értesítést vált ki. A hiányzó rész: teljes tranzakciós e-mail életciklus — hibabejelentés státuszfrissítések, közgyűlési meghívók (Ptk. 5:84 alapján jogilag 8 nappal előre kötelező), havi közös költség kimutatások, hátralékértesítők. E-mail nélkül a PanelLakó nem szolgálhatja ki azokat a lakókat, akik nem ellenőrzik az alkalmazást naponta — ami Magyarországon az 50 év felettiiek többsége.

Az e-mail a legszélesebb elérési kommunikációs csatorna a KKE ingatlankezelésben. A KSH (2024) szerint a 45-64 éves magyarok 78%-a naponta használ e-mailt, vs. csak 34%-uk push-kompatibilis appot. A jogilag kötelező kommunikációhoz (közgyűlési meghívók, hátralékértesítők) az e-mail az egyetlen jogi érvénnyel auditálható kézbesítési csatorna.

Technikai megközelítés: a meglévő ``lib/email.ts`` bővítése tipizált ``EmailEvent`` enummal és React Email sablonok létrehozása: `ticket-status-change`, `assembly-invitation`, `monthly-statement`, `arrears-notice`, `document-shared`. Minden sablon bekötése a megfelelő Server Actionhoz, minden küldés naplózása az ``audit_logs``-ban.

3. PONT — MEGVALÓSÍTÁSI PROMPT

```
1. `lib/email.ts` bővítése: `EmailEventType` enum hozzáadása; `sendTypedEmail(event, to, data)` diszpécser.  
2. `lib/email-templates/ticket-status-change.tsx`: 'Hibabejelentés frissítve: {title}'; régi → új státusz, épület cím, link.  
3. `lib/email-templates/assembly-invitation.tsx`: Ptk. 5:84 kompatibilis; tartalmazza az épület nevét, dátumát, helyszínét, napirendi pontokat, meghatalmazási instrukciókat.  
4. `lib/email-templates/monthly-statement.tsx`: albetét szám, hónap, terhelés, befizetés, egyenleg, PDF letöltési link.  
5. `app/actions/tickets.ts` `updateTicketStatus()`-ban: `sendTypedEmail('ticket_update', ...)` hívás.  
6. `app/actions/meetings.ts` `sendAssemblyInvitation()`-ban: `sendTypedEmail('assembly_invitation', allUnitOwnerEmails, ...)` + `audit_logs` bejegyzés.  
7. `app/actions/notifications.ts` `sendMonthlyStatements()`: minden e-maillal rendelkező albetéthez havi kimutatás küldése.  
8. `RESEND_API_KEY` beállítása Vercel projekt környezeti változóban; `panellako.hu` domain hozzáadása a Resend küldési domainjekhez.
```

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Kommunikációs elérés	Csak alkalmazás (30% napi aktív) → Alkalmazás + E-mail (78% napi elérés)
Jogi megfeleléség	Ptk. 5:84 közgyűlési meghívó auditnyommal teljesítve
Lakói engagement	+60-80% elérés push-only-hoz képest az 50 év felettieknél
Értékelési hatás	+180 000-400 000 € (megtartás + megfeleléségi feloldás)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€180k-€400k**5. PONT — ÚJRAGENERÁLÁSI PROMPT**

Senior Next.js 14 + Resend e-mail fejlesztőként valósítsd meg a teljes tranzakciós e-mail csomagot a Panellakóban (v0.9.23, /home/user/panellako). Meglévő: `lib/email.ts`, `app/actions/tickets.ts`, `app/actions/meetings.ts`, `app/actions/finance.ts`. Valósítsd meg: (1) `EmailEventType` diszpécser, (2) React Email sablonok ticket-status-change, assembly-invitation, monthly-statement, arrears-notice, document-shared, (3) sablonok bekötése Server Actionökhöz, (4) audit_logs bejegyzések, (5) RESEND_API_KEY beállítás.

RANG 7
Környezeti intelligencia dashboard — SEO-ból termékkonverzió motor

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A Panellakó jelentős beruházást tett a környezeti adatinfrastruktúrába: levegőminőség (`components/air-quality-section.tsx`), hősziget elemzés (`components/heat-island-dashboard-client.tsx`), zajszennyezés (`components/noise-dashboard-client.tsx`), területhasználati térképek, kerékpáros útvonalak, zöld pontszám, élhetőségi panel és közszolgáltatások. Ez valódi versenyelőny: egyetlen magyarországi PropTech versenytárs sem rendelkezik valós épület-szintű környezeti elemzéssel.

A SEO tartalomklaszter (`app/levegominoseg-budapest/`, `app/klimakockazat-epuleteknel/`, `app/zajszennyez-es-budapest/`, `app/zold-tarsashaz/`) már generál organikus forgalmat ezekre az elemzési cikkekre. A konverziós út SEO cikktől → termék regisztrációig → épület környezeti dashboardig részlegesen kiépített, de nem teljesen optimalizált. Lehetőség: nyilvános 'Épület Környezeti Pontszám' oldal lead mágnesként, premium funkció upsell, és környezeti pontszámok B2B értékesítésben önkormányzatokhoz.

Technikai megközelítés: nyilvános `/epulet/{buildingId}/kornyezet` oldal létrehozása bejelentkezés nélkül korlátozott környezeti összefoglalóval (hősziget kockázat, zöld pontszám, levegőminőségi index) mint lead mágnes. Bejelentkezett felhasználóknak: historikus trendek, peer benchmarking, akcióképes fejlesztési javaslatok, EU EPBD 2024/1275/EU megfeleléségi jelentés.

3. PONT — MEGVALÓSÍTÁSI PROMPT

1. Hozd létre: `app/epulet/[buildingId]/kornyezet/page.tsx` – nyilvános környezeti összefoglaló (nincs auth); `building\_env\_score` táblából; hősziget kockázat kártya, zöld pontszám mérőóra, levegőminőségi index.

2. 'Regisztráld a teljes elemzésért' CTA gomb hozzáadása → `/ingyenes-proba?source=env\_score&building={buildingId}`.

3. `app/w/[buildingId]/(subpages)/kornyezet/` oldalon: historikus trenddiagramok (Recharts LineChart) az `air\_quality\_readings` táblából.

4. Peer benchmarking: `getDistrictAverageScores(districtCode)` Server Action – épület env pontszámának összehasonlítása a kerület mediánjával.

5. `components/env-improvement-recommendations.tsx` létrehozása: szabályalapú javaslatok (pl. alacsony zöld pontszám → 'zöldtető jogosult').

6. EU EPBD megfeleléségi szekció: EPC osztály megjelenítése, link az energetikai tanúsítvány cikke.

7. `app/sitemap.ts` bővítése nyilvános `/epulet/[buildingId]/kornyezet` oldalakra.

8. PostHog esemény: `env\_score\_page\_viewed` konverziós tölcser követéshez.

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Lead mágnes konverzió	Nyilvános env pontszám oldal → ingyenes próba CTA
SEO-ból termék tölcser	Környezeti cikk forgalom → aláírt épületek
Versenyelőny	Egyetlen épület-szintű környezeti elemzés a HU PropTechben
Önkormányzati/lakásszövetkezeti B2B feloldás	EU EPBD megfeleléségi jelentés = kormányzati értékesítés

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€150k-€340k

5. PONT — ÚJRAGENERÁLÁSI PROMPT

Senior Next.js 14 + Supabase fejlesztőként tervezd meg a Panellakó környezeti intelligencia dashboardját (v0.9.23, /home/user/panellako). Meglévő: `building\_env\_score` tábla, `air\_quality\_readings` tábla, `components/air-quality-section.tsx`, `components/heat-island-dashboard-client.tsx`, `components/green-score-dashboard-client.tsx`. Tervezd meg: (1) nyilvános lead mágnes oldal, (2) historikus trenddiagramok, (3) peer benchmarking Server Action, (4) fejlesztési javaslatok komponens, (5) EU EPBD megfelelési szekció, (6) sitemap.ts bejegyzés, (7) PostHog konverziós események.

RANG 8

SSR auth keményítés + Middleware route védelem

2. PONT – LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A Panellakó hitelesítése Supabase Auth-ot használ, de a governance szabályok (`CLAUDE.md`, `AI\_EXECUTION\_PROMPTS.md`) megkövetelik, hogy az összes `/w/[buildingId]/` route szerver-oldali védelemmel rendelkezzen. A jelenlegi állapot kliens-oldali munkamenet-ellenőrzésre támaszkodik, amely elavult lehet (`getSession()` lokális gyorsítótárból olvas). Egy érzékeny pénzügyi adatokat, hátralék-nyilvántartásokat és lakói személyes adatokat (GDPR-védett) kezelő termék számára ez biztonsági rés.

A magyarországi GDPR végrehajtás felerősödött, miután a NAIH (Nemzeti Adatvédelmi és Információszabadság Hatóság) 2024-ben kötelező érvényű iránymutatást adott ki a lakáskezelő szoftverekre. A szerver-oldali munkamenet-érvényesítés nélkül lakói személyes adatokat feltáró eszközök GDPR 83(4) cikke alapján a globális éves forgalom akár 4%-ának megfelelő bírsággal szembesülhetnek. A NAIH-ellenőrzés lelet egy PropTech startup számára egzisztenciális fenyegetés.

Technikai megközelítés: `@supabase/ssr` telepítése (a Next.js 14 hivatalos Supabase auth könyvtára), `lib/supabase/server.ts` létrehozása cookie-alapú szerver klienssel, `middleware.ts` létrehozása a repo gyökerében, amely minden `/w/\*\*` route-ra meghívja az `updateSession()`-t. Az összes `getSession()` hívás lecserélése `getUser()`-re szerver komponensekben.

3. PONT – MEGVALÓSÍTÁSI PROMPT

```
1. `npm install @supabase/ssr` – hozzáadás a `package.json`-hoz.  
2. `lib/supabase/server.ts` létrehozása: `createServerClient` cookie-alapú konfigurációval.  
3. `middleware.ts` létrehozása a repo gyökerében: `updateSession(request)` meghívása `(w|app|billing|superadmin)/**` route-okra; hitelesítetlen kérések átirányítása `/login`-ra.  
4. `config = { matcher: [...] }` hozzáadása a middleware-hez statikus eszközök kizárásával.  
5. Az összes `createClient()` lecserélése `createSupabaseServerClient()`-re az `app/w/[buildingId]/**` szerver komponensekben.  
6. Az összes `getSession()` lecserélése `getUser()`-re – `const { data: { user } } = await supabase.auth.getUser(); if (!user) redirect('/login');` minden védett oldal tetejére.  
7. Az összes `app/actions/*.ts` Server Actionban: `const { data: { user } } = await supabase.auth.getUser(); if (!user) throw new Error('Unauthorized');` hozzáadása minden DB mutáció elé.  
8. Teszt: manuálisan töröld a Supabase auth cookie-t a böngésző DevTools-ban; ellenőrizd az átirányítást.
```

4. PONT – ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Biztonsági állapot	Cache auth → Szerver-ellenőrzött auth minden kérésnél
GDPR megfeleléség	NAIH lakáskezelési iránymutatás → vállalati szerződési jogosultság
Önkormányzati/vállalati deal feloldás	Biztonsági kifogás eltávolítva a B2B értékesítési ciklusból
Értékelési hatás	+130 000-300 000 € (bizalmi prémium + deal feloldás)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€130k-€300k

5. PONT — ÚJRAGENERÁLÁSI PROMPT

Senior Next.js 14 + Supabase SSR fejlesztőként valósítsd meg az auth keményítést a Panellakóban (v0.9.23, /home/user/panellako). Az app Supabase Auth-ot használ, de kliens-oldali munkamenet-ellenőrzéssel. CLAUD E.md governance megköveteli a szerver-oldali védelmet az összes `/w/[buildingId]/` route-ra. Valósítsd meg: (1) `@supabase/ssr` telepítés, (2) `lib/supabase/server.ts` cookie-alapú szerver klienssel, (3) `middleware.ts` `updateSession`-nel, (4) `getSession()` → `getUser()` csere, (5) Server Action auth ellenőrzések.

RANG 9

Lakói önkiszolgáló portál — Mobil PWA + Push értesítés mélyítés

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A Panellakóban van PWA manifest, service worker (`supabase/functions/send-push/`) és web-push infrastruktúra. Ami hiányzik: dedikált lakói önkiszolgáló portál — mobil-optimalizált nézet, ahol a lakók (1) hibabejelentéseket küldhetnek fotófeltöltéssel, (2) megtekinthetik albetétük fizetési állapotát, (3) épületi dokumentumokat böngészhetnek (SZMSZ, házirendek) a kezelő-fókuszú dashboard nélkül, és (4) közgyűlési meghívókra válaszolhatnak és meghatalmazást adhatnak.

A lakói engagement szorzóan hat a kezelői megtartásra: egy kezelő, aki azt mondhatja 'a lakóink naponta használják a Panellakót', sokkal kevésbé valószínű, hogy lemorzsolódik. A magyarországi piacon a fájdalompont a WhatsApp csoport — minden épület jelenleg WhatsApp csoportot használ, amely nem archiválható, nem auditálható. A Panellakó PWA-ja (Androidon és iOS-en egyaránt telepíthető böngészőből) plusz web-push felváltja a WhatsApp csoportokat, miközben strukturált adatokat ad hozzá.

Technikai megközelítés: `/portal` route alrendszer létrehozása a lakói élményhez, elkülönítve a kezelő-fókuszú `/w/[buildingId]/` route-októl. A portál ugyanazokból a Supabase táblákból olvas, de lakói RLS házirendeket használ. Kulcs komponensek: `components/resident-ticket-form.tsx` (kamera elfogással), `components/resident-balance-card.tsx`, `components/resident-document-list.tsx`, `components/resident-assembly-rsvp.tsx`.

3. PONT — MEGVALÓSÍTÁSI PROMPT

1. `app/portal/[buildingId]/page.tsx` létrehozása: lakói főoldal — épület neve, legutóbbi értesítés, köze lgő közgyűlés dátuma.
2. `app/portal/[buildingId]/hiba/page.tsx`: hibabejelentés `- 3. `app/portal/[buildingId]/egyenleg/page.tsx`: albetét fizetési státusz az `unit_ledger_view`-ből.
- 4. `app/portal/[buildingId]/dokumentumok/page.tsx`: csak-olvasási dokumentumlista Supabase Storage aláírt URL-lel.
- 5. `app/portal/[buildingId]/kozgyules/page.tsx`: RSVP form (részt vesz/nem/meghatalmazott); meghatalmazás feltöltés.
- 6. `public/manifest.json` frissítése: `start\_url: '/portal/'` hozzáadása.
- 7. `supabase/functions/send-push/index.ts`-ben: `resident\_announcement` és `ticket\_resolved` értesítési típusok.
- 8. 'Meghívó küldése lakóknak' gomb a kezelői nézetben: épület-specifikus portal URL generálása és Resend e-mail küldése.

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Lakói DAU/WAU épületenként	0 → cél: az albetéttulajdonosok 30-50%-a hetente aktív
WhatsApp csoport helyettesítés	Strukturált, auditálható kommunikációs csatorna
Kezelői megtartás	+25-35% — aktív lakókkal rendelkező kezelők 2-3× kevésbé morzsolódnak le
Értékelési hatás	+100 000-240 000 € (engagement mélység = megtartás = LTV szorzó)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€100k-€240k**5. PONT — ÚJRAGENERÁLÁSI PROMPT**

Senior Next.js 14 + mobil UX fejlesztőként tervezd meg a Panellakó lakói önkiszolgáló portálját (v0.9.23, /home/user/panellako). Megelevő: PWA manifest, `supabase/functions/send-push/`, `app/actions/tickets.ts`, `app/actions/meetings.ts`, `documents` bucket, `unit\_ledger\_view`. Tervezd meg: (1) `app/portal/[buildin gId]/page.tsx` lakói főoldal, (2) hibabejelentési form kamera elfogással, (3) albetét egyenleg kártya, (4) dokumentumböngésző, (5) RSVP form, (6) manifest.json frissítés, (7) lakói push értesítési típusok, (8) kezelői meghívó gomb. Mobil-first, 44px érintési célpontok.

PANG 10

PostHog termék-analitika — Konverziós tölcsér + funkció-használat instrumentálás

2. PONT — LEÍRÁS ÉS PIACI BIZONYÍTÉKOK

A PanelLakóban telepítve van a PostHog (``next.config.mjs`` CSP tartalmazza az ``eu.posthog.com``-ot), de az instrumentálás valószínűleg minimális — alap oldalmegtekintések inkább, mint strukturált esemény-taxonómia a teljes termék tölcsér lefedéséhez. A jelenlegi növekedési szakaszban (v0.9.23, valós számlázással) a legmagasabb leverage-sú analitikai munkák: a trial-to-paid konverziós tölcsér részletes kiesési pontokkal való feltérképezése, melyik funkciók korrelálnak legerősebben a konverzióval, és kohort megtartási görbe építése a befektetők számára.

Adatvezérelt termékdöntések értékelési szorzót jelentenek: a VC-k és stratégiai felvásárlók 20-40%-kal többet fizetnek SaaS vállalkozásokért bizonyítható termék-vezérelt növekedési mutatókkal (felhasználói aktiválási arány, funkció-adoptálási arány, kohort megtartási görbék). Az SEO beruházás (v0.9.11-v0.9.23 sprint sorozat) organikus forgalmat generál — a PostHog tölcsérek megmutatják, melyik tartalmak konvertálnak próbaidőszakra.

Technikai megközelítés: ``PanelLakoEvent`` TypeScript enum definiálása a teljes termékútra (20-30 esemény). Tölcsér szakaszok szerint csoportosítva: (1) Megszerzés; (2) Aktiválás; (3) Bevétel; (4) Megtartás. ``lib/analytics.ts``-ben tipizált ``trackEvent`` wrapper-rel.

3. PONT — MEGVALÓSÍTÁSI PROMPT

```
1. `lib/analytics.ts` létrehozása: `trackEvent(event: PanelLakoEvent, properties?)` wrapper PostHog `posthog.capture()`-ral.  
2. `PanelLakoEvent` enum definiálása: megszerzési események, aktiválási események, bevételi események, megtartási események (összesen 30).  
3. `trackEvent('trial_cta_clicked', {source: 'hero'|'pricing'|'env_score'})` hozzáadása az összes CTA gombhoz a nyilvános oldalakon.  
4. Aktiválási követés: `createTicket()`-ben `trackEvent('ticket_submitted', {building_id, ai_triage_enabled})`.  
5. `trackEvent('trial_converted', {plan, unit_count, building_count})` a Stripe webhook route-ban.  
6. PostHog Dashboard létrehozása: 'Trial Tölcsér', 'Funkció Adoptálási Mátrix', 'Kohort Megtartás'.  
7. PostHog Feature Flags A/B teszteléshez: `onboarding_flow_v2` flag.  
8. `posthog.identify(user.id, {plan, building_count, unit_total})` az `app/w/[buildingId]/page.tsx`-ben.
```

4. PONT — ÉRTÉKNÖVEKEDÉSI BECSLÉS

Metrika	Hatás
Befektetői készség	Kohort megtartási görbék → Series A adatszoba-kész
Tartalom-to-trial konverzió láthatóság	SEO organikus → próba CTA forrás-attribúció
Termékdöntések	Funkció-használati adatok → megalapozott roadmap prioritizálás
Értékelési hatás	+80 000-200 000 € (adatvezérelt SaaS 20-40% értékelési prémiumot parancsol)

BECSÜLT ÉRTÉKNÖVEKEDÉS

+€80k-€200k

5. PONT — ÚJRAGENERÁLÁSI PROMPT

Senior termék-analitika fejlesztőként valósítsd meg a teljes PostHog instrumentálást a Panellakóban (v0.9.23, /home/user/panellako). PostHog telepítve van (CSP: eu.posthog.com). Valósítsd meg: (1) `lib/analytics.ts` `PanellakoEvent` enummal (30 esemény), (2) `trackEvent` wrapper, (3) CTA követés nyilvános oldalakon, (4) Server Action követés, (5) Stripe webhook követés, (6) PostHog identify felhasználói tulajdonságokkal, (7) Feature Flag A/B teszteléshez.

TELJES ÉRTÉKMÁTRIX — Az összes Növekedési Kezdeményezés

Ran g	Kezdeményezés	Becsült Hatás
1	Multi-épület portfólió dashboard — Közös képviselői skálázási architektúra	+€450k-€900k
2	Teljes Stripe előfizetési életciklus — Próba → Fizetős → Lejárt → Lemondás	+€380k-€800k
3	AI hibabejelentés triage + kivitelező irányítás — Versenyelőny Claude API-val	+€320k-€680k
4	Automatizált közgyűlési jegyzőkönyv generátor — Közgyűlési Jegyzőkönyv PDF	+€250k-€550k
5	Teljes pénzügyi főkönyv — Kettős könyvviteli közös költség könyvelés	+€220k-€480k
6	Tranzakciós e-mail csomag Resend-del — Teljes kommunikációs életciklus	+€180k-€400k
7	Környezeti intelligencia dashboard — SEO-ból termékkonverzió motor	+€150k-€340k
8	SSR auth keményítés + Middleware route védelem	+€130k-€300k
9	Lakói önkiszolgáló portál — Mobil PWA + Push értesítés mélyítés	+€100k-€240k
10	PostHog termék-analitika — Konverziós tölcsér + funkció-használat instrumentálás	+€80k-€200k
TELJES KOMBINÁLT POTENCIÁL		+€2.26M-€4.89M

Kiindulási értékelés: €400k-€2.2M

→ Célértékelés: €2.66M-€7.09M (3-5x)